2 Tutorial 1: Reading in our data

In the first Lab, your tutors will be assisting you to do the initial set up work (Tutorial Exercises 1-2) so this chapter will be shorter. As a summary of the steps:

1. **We will set up R and R Studio**. Your tutors will help you do any set up work here.
2. **Set up an R Project:** Once R and R Studio has been set up, your tutors will show you how to set up an R project and explain why this is important in order to keep everything in one place.
3. **Download your data:** After that,your tutors will go through downloading the Airbnb data and then transferring to your R Studio project folder.
4. **Install the tidyverse() package:** You will then need to install the tidyverse package, which we will use on the course, You will need to do this in the console. And again, your tutors will explain this.
5. **Open up an R Script:** You can then open up an R script. We will use an R Script in this lab. In later labs, we will transition to something called R Markdown.

We will pick up at **4. Install the tidyverse() package**

The console is the left pane: and is the first of the three tabs (Console, Terminal, Background Jobs). See if you can locate it: it's where the output is shown: for example, when you give a command in R, the result (or output) is shown in the console.

We can use the console for things we only need to do once: like installing a package. One a package is installed, that's it. We only need to do it once. We can also use the console for testing things out to see if they work or for doing quick calculations. So the Console has two functions:

1. Showing us the output of the commands we input
2. A temporary space to do one off tasks

Here is the code to install the tidyverse package:

```
# you must run this in the console not the script
# install.packages("tidyverse")
```

It can take up to a minute to install the tidyverse since it contains a lot of packages with in.

Remember: to install a new package, use the console.

5. Open up an R Script:

We will now open up an R script. Go up to the File Menu at the top left hand of R Studio. Then:

New File > R Script

If you are unsure, ask your Lab Colleagues or Tutors. Don't suffer in silence.

You will see we know have 4 Window Panes instead of 3 as previously.

We use scripts to save our work. We can then easily run all the commands from the previous week and pick up where we left off. Scripts also make our work transparent and reproducible, and easy to share with others.

Although we have installed the tidyverse package, we now need to load it: type the following into your R Script and then `Run` the line of code: it's the button at the top. Ask your colleagues or Tutors if you can;t find it. A handy keyboard shortcut is: `Ctrl + Enter`

```
# load the tidyverse package
library(tidyverse)
```

We will use this package (or rather the packages contained within it) for a variety of things: from reading in our data to plotting.

2.0.1 Reading in the data and inspecting it

Next, we will read in our Airbnb data:

There are two steps to this:

1. We use the `read_csv()` function. This takes the name of the data as the argument: `"listings_edinburgh.csv"`. Note this includes the file extension i.e. `.csv`. See the code below for reference: this reads in our Airnbnb data.

   ```
   read_csv("listings_edinburgh.csv")
   ```

   ```
   ## Rows: 14014 Columns: 10
   ## ── Column specification ────────────────────────────────────────────────────────
   ## Delimiter: ","
   ## chr (4): name, neighbourhood_cleansed, property_type, room_type
   ## dbl (6): accommodates, bathrooms, bedrooms, beds, price, review_scores_rating
   ##
   ## ℹ Use `spec()` to retrieve the full column specification for this data.
   ## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
   ```

   ```
   ## # A tibble: 14,014 × 10
   ##    name       neighbourhood_cleansed property_type room_type accommodates bathrooms bedrooms  beds
   ##    <chr>      <chr>                  <chr>         <chr>            <dbl>     <dbl>    <dbl> <dbl>
   ##  1 Georgian … Old Town, Princes Str… Apartment     Entire h…            2       1          1     1
   ##  2 Cool cent… Meadows and Southside  Loft          Entire h…            4       1.5        2     2
   ##  3 Edinburgh… Joppa                  Apartment     Entire h…            2       1          0     2
   ##  4 Double ro… South Leith            Condominium   Private …            2       1          1     1
   ##  5 City flat… Canongate, Southside … Condominium   Private …            2       1          1     1
   ##  6 Large Sun… Old Town, Princes Str… Apartment     Private …            3       1          1     2
   ##  7 Superb Ci… New Town West          Apartment     Entire h…            5       1          2     3
   ##  8 The Four … Tollcross              Boat          Entire h…            5       1.5        3     2
   ##  9 Lovely ce… Hillside and Calton H… Apartment     Entire h…            6       1          4     5
   ## 10 Beautiful… Deans Village          Apartment     Entire h…           10       2          4     7
   ## # ℹ 14,004 more rows
   ## # ℹ 2 more variables: price <dbl>, review_scores_rating <dbl>
   ```

2. But we need to assign this to an object variable. The code abovely reads in the data temporarily. When we close our R session down, it will disappear and we will need to do it all over again. We want to make this permanent. So we need to store it in a variable.

   To assign the data to a variable, we use the assignment operator `<-`. And we need to give our variable a name. We can call it anything. Although you will need an underscore if you use more than one word. We're using the Airbnb listings, so I'll keep it simple and call it `listings`. It doesn't matter as long as you

know what it refers to. If you keep it simple, it makes it easier to type out also.

So know we add the second part: assigning the first part: `read_csv("listings_edinburgh.csv"` to a variable.

3. To do so, type the following into your R Script and `Run` the line of code:

```
# read in the dataset
listings  <- read_csv("listings_edinburgh.csv")
```

```
## Rows: 14014 Columns: 10
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## chr (4): name, neighbourhood_cleansed, property_type, room_type
## dbl (6): accommodates, bathrooms, bedrooms, beds, price, review_scores_rating
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

You may now notice that in the top right Window Pane (the Environment), we now have an object called `listings`. So we know we've done this right.

Now we have this object (i.e. our Airnbnb listings dataset), we can perform operations on it. The first thing we want to do is to inspect our data.

As before, type the following commands into your R Script and run each one.

```
# look at your dataset:
View(listings)
glimpse(listings)
```

```
## Rows: 14,014
## Columns: 10
## $ name                   <chr> "Georgian Boutique Apt City Centre", "Cool central Loft, sleeps 4…
## $ neighbourhood_cleansed <chr> "Old Town, Princes Street and Leith Street", "Meadows and Southsi…
## $ property_type          <chr> "Apartment", "Loft", "Apartment", "Condominium", "Condominium", "…
## $ room_type              <chr> "Entire home/apt", "Entire home/apt", "Entire home/apt", "Private…
## $ accommodates           <dbl> 2, 4, 2, 2, 2, 3, 5, 5, 6, 10, 2, 4, 3, 2, 2, 4, 2, 4, 10, 5, 2, …
## $ bathrooms              <dbl> 1.0, 1.5, 1.0, 1.0, 1.0, 1.0, 1.0, 1.5, 1.0, 2.0, 1.0, 2.0, 1.0, …
## $ bedrooms               <dbl> 1, 2, 0, 1, 1, 1, 2, 3, 4, 4, 1, 1, 1, 1, 1, 2, 1, 1, 5, 1, 1, 1,…
## $ beds                   <dbl> 1, 2, 2, 1, 1, 2, 3, 2, 5, 7, 1, 1, 1, 1, 1, 2, 1, 1, 5, 2, 1, 1,…
## $ price                  <dbl> 80, 115, 46, 32, 100, 71, 175, 150, 139, 190, 85, 64, 79, 60, 55,…
## $ review_scores_rating   <dbl> 99, 92, 94, 93, 98, 97, 100, 92, 96, 99, 77, 98, 98, 96, 98, 89, …
```

The `View()` function gives us a more familiar 'spreadsheet' style view and opens up a new window. You can also sort and filter as you can in Excel as well. Note it has a capital 'V'. Have a look at the data as you would a spreadsheet to get an initial impression of the data. Play about with the sort and filter buttons or ask your Tutor to help.

The `glimpse()` function gives us two pieces of information that `View()` function does not:

1. **The dimensions:** this means how many rows and columns we have.
2. **The variable data types:** it gives us the *data types* of the variables. This is critically important to know (as emphasised in Lecture 1). R gives us these as short codes:

    - <chr> is a character data type: text like a name: "Edinburgh Old Town"
    - <fct> is a factor variable: factor or *categorical* variables, which have categories like `room_type`, which has three categories of room: Entire Home/Appt, Shared Room and Private Room

We have also have two kinds of *numeric* variables:

- <dbl> which is a *double* type of numeric variable: think decimal for double e.g. 1.2, 2.5, 3.87.
- <int> which is an *integer* type of numeric variable: think whole numbers e.g. 1, 2, 3

Question: How many rows and columns do we have?

Our *rows* are our observations: our Airbnb listings

Our *columns* are our variables: the different ways we measure those Airbnb listing e.g. by how many rooms they have (`room_type`), what reviews scores they got (`review_scores_rating`) and so on.

- So how many Airbnb listings (rows) do we have in our dataset?
- Also, how many variables do we have?

Question: How many data types do we have?

Now, let's look at our individual variables and their data types.

What data type are the following variables:

- `property_type`: character, factor or numeric (either `dbl>` or `<int>)`?
- `review_scores_rating:` character, factor or numeric?
- `price`: character, factor or numeric?
- `neighbourhood_cleansed`: character, factor or numeric?
- `bathrooms:` character, factor or numeric?

Remember a numeric variable can be either `<dbl>` or `<int>`

Check your answers with your Lab colleagues or Tutors to make sure you are getting this right. Understanding your variable *data types (<chr>, <fct> etc)* is a really important thing to grasp.

Question: do these variables look correct to you?

When we look at `room_type` for example, this should be a *categorical* variable. R calls this a factor variable `<fct>`. We can deduce this since it has three *categories* (Entire Home/Apt; Shared Room; Private Room). It should not be character `<chr>` and be treated as mere text. R needs to know this is a factor vauiable. And we need R to recognise and read in the variables as having three distinct categories: Entire Home/Apt; Shared Room; Private Room. That way our c.14,000 Edinburgh Airbnb listings will each be classified into each of these three categories.

But *why* are we needing to do this in the first place? R can read data in any format: webpages, datasets, Excel spreadsheets, SPSS files. Virtually anything, which is one of its strengths. But because of this vast array of formats it can read in, it has one downside: R can read in some variables in the wrong format. This is *often the case for factor variables* and we need to always check these carefully. Here were have no documentation so need to use our common sense but you will normally have documentation you can check (and which is supplied when you download your data) i.e. *is* this a factor variable? Note: it will more probably be referred to by one if its non-R specific synonyms: nominal or categorical.

So we need to figure out: what variable(s) *should* have been read in as a factor variable but wasn't/weren't? One we locate these, we need to change them: from whatever they are currently (e.g. `<chr>` or `<int>`) to `<fct>`.

To change a data type is called *coercion*. We need to change our `room_type` from character `<chr>` data type to factor `<fct>`. Let's do this now:

Note: the numeric variables (Bedrooms, Bathrooms etc) all look fine. One technical point: remember <int> are whole numbers and <dbl> are decimals or fractions.

- `bathrooms, bedrooms, beds` are `<dbl>` but they should really be `<int>` or whole numbers since we cannot have 0.5 or part/fraction of a `bathroom` or `bed`.
- Similarly, the variables for `price` and `review_scores_rating` are <dbl>: we'd expect this since price, for example, is often a decimal or fraction e.g. $1.25, £2.30. But when we look at their values, they are whole numbers (no decimal points).

We'll leave these as they are: the main thing is that they are recognised as *numeric* (either `<dbl>` or `<int>`). And then we won't have any problems further down the line. But if we were going to be *technically correct*, we would want to coerce these also. We'll leave it for the Lab Tutorials because we have so much to get through but feel free to try this on your own and/or ask your Tutors for help later on.

2.0.2 Coercing variables

We will use the `mutate()` function to coerce variables:

```
# change room_type from character to factor
listings <- mutate(listings,
                   room_type = factor(room_type))
```

Let's look at the syntax of the mutate function:

`{r} # mutate (dataset,} # result you want = original that you are going to change)`

We supply a function with *arguments*. The first is the dataset: we have called ours listings, when we assigned to an object variable. Within that datset, is a variable called `room_type`. I am going to coerce this to factor `<fct>` using the `factor()` function. Then assign the result back to room_type, in effect overwriting it: `room_type = factor(room_type)`

You will notice we have used a function within a function: we have used the `factor()` function *within* the `mutate()` function. R has this flexibility so we can do this. But note, having used two functions, we need to close the brackets *twice* therefore. Don't forget this or you will get an error.

More advanced (optional): the `factor` function is doing the actuall coercion here: changing the data type to `<fct>`. The `mutate` function can be seen as a 'wrapper' function: it enables us to coerce more than one variable at a time (as you'll see below). This is very useful as we can often have a lot of variables to coerce.

OK, let's `run` the code below (or `Crtl + Enter`):

`{r} # change room_type from character to factor listings <- mutate(listings, room_type = factor(room_type))}`

And let's check to see if it worked. You can see a pattern here: when we do something, we need to check to see if has worked like we intended.

To do so, we can use the `glimpse()` command again:

```
glimpse(listings)
```

```
## Rows: 14,014
## Columns: 10
## $ name                   <chr> "Georgian Boutique Apt City Centre", "Cool central Loft, sleeps 4…
## $ neighbourhood_cleansed <chr> "Old Town, Princes Street and Leith Street", "Meadows and Southsi…
## $ property_type          <chr> "Apartment", "Loft", "Apartment", "Condominium", "Condominium", "…
## $ room_type              <fct> Entire home/apt, Entire home/apt, Entire home/apt, Private room, …
## $ accommodates           <dbl> 2, 4, 2, 2, 2, 3, 5, 5, 6, 10, 2, 4, 3, 2, 2, 4, 2, 4, 10, 5, 2, …
## $ bathrooms              <dbl> 1.0, 1.5, 1.0, 1.0, 1.0, 1.0, 1.0, 1.5, 1.0, 2.0, 1.0, 2.0, 1.0, …
## $ bedrooms               <dbl> 1, 2, 0, 1, 1, 1, 2, 3, 4, 4, 1, 1, 1, 1, 1, 2, 1, 1, 5, 1, 1, 1,…
## $ beds                   <dbl> 1, 2, 2, 1, 1, 2, 3, 2, 5, 7, 1, 1, 1, 1, 1, 2, 1, 1, 5, 2, 1, 1,…
## $ price                  <dbl> 80, 115, 46, 32, 100, 71, 175, 150, 139, 190, 85, 64, 79, 60, 55,…
## $ review_scores_rating   <dbl> 99, 92, 94, 93, 98, 97, 100, 92, 96, 99, 77, 98, 98, 96, 98, 89, …
```

Yes, `room_type` has now changed to `<fct>` so we've done this right.

We also have property type to coerce as well. Rather than do them all one by one we can combine our code into one line:

```
# change both at same time ☺
listings <- mutate(listings,
                   room_type = factor(room_type),
                   property_type = factor(property_type))
```

If you look back at the Tutorial 1 Exercises you will see they have now all been answered:

1. The first task is to install R: https://cran.r-project.org/index.html. And then, install R Studio
2. Set up a new project that you can use to work on your tutorial exercises (File > New Project). Download the airbnb data from Moodle and save it in the same folder you used for your project.
3. Set up a new R script which you can use to complete the exercises.
4. Load the data using the *read_csv* function.
5. Use the *glimpse()* function to look at the data.
6. Convert the room_type variable into a factor variable. It has three levels: *Entire home/apt*, *Private room* and *Shared room*.

The first three questions related to *Setting Up R*. Make sure you have now done this correctly. This is important for the rest of the course.

Questions 4-6 related to *Reading in and checking our data* and making any changes if it was not correct. We *always* need to do this with any data we read in. And you will need to do it with your data project. We cannot trust it is automatically correct. And this is often the case with factor `<fct>` variables.